
PROJECT PROPOSAL

Heart Disease Prediction API

Machine Learning-Based REST API for Healthcare Decision Support

Prepared by

Abdullahi Abdiweli Adam

July 2026

1. Certificate Name

Abdulaahi Abdiweli Adam

2. Project Title and Description

Title: Heart Disease Prediction API

Description

Heart disease is one of the leading causes of death worldwide, and early detection can significantly improve treatment outcomes. Hospitals and healthcare professionals collect a large amount of patient data during routine examinations, but identifying patients at risk can be challenging without automated tools.

This project aims to build a machine learning model that predicts whether a patient has heart disease using medical information such as age, blood pressure, cholesterol level, chest pain type, and heart rate. The final deliverable will be a REST API that accepts patient information as JSON and returns a heart disease prediction with a probability score.

This project demonstrates how machine learning can support healthcare professionals by providing a fast and consistent decision-support tool.

3. Problem Type

Classification — Binary Output

- Heart Disease (Positive Class)
- No Heart Disease (Negative Class)

Target Variable: *target*

- ▶ 1 = Heart Disease
- ▶ 0 = No Heart Disease

This is a supervised machine learning problem because the dataset contains historical patient records with known diagnosis outcomes. The model will learn patterns from labeled data to make predictions on new, unseen patient records.

4. Dataset

SOURCE <https://www.kaggle.com/mfarhaannazirkhan/heart-dataset>

SIZE 2,182 rows × 14 columns

TARGET indicates whether the patient has heart disease (1) or does not have heart disease (0)

Feature Overview

Feature	Description
age	Patient's age (years)
sex	Patient's gender
cp	Chest pain type
trestbps	Resting blood pressure (mm Hg)
chol	Serum cholesterol level (mg/dl)
fbs	Fasting blood sugar > 120 mg/dl
restecg	Resting electrocardiographic results
thalachh	Maximum heart rate achieved
exang	Exercise-induced angina
oldpeak	ST depression induced by exercise
slope	Slope of the peak exercise ST segment
ca	Number of major vessels colored by fluoroscopy
thal	Thalassemia result

Preprocessing Pipeline

- ▶ Check for missing values and handle them using appropriate imputation strategies
- ▶ Remove duplicate records to ensure data quality and integrity
- ▶ Perform exploratory data analysis (EDA) to understand feature distributions and correlations
- ▶ Scale numerical features using StandardScaler for normalization and convergence stability
- ▶ Encode categorical values using one-hot or ordinal encoding as appropriate
- ▶ Split the dataset into training and testing sets using an 80/20 stratified split to maintain class balance

5. Algorithms I Plan to Train

Three algorithms will be trained and evaluated to identify the best-performing model. Logistic Regression and Random Forest were covered during the bootcamp, while Support Vector Machine (SVM) will be researched using scikit-learn documentation and tutorials.

#	Algorithm	Rationale
1	Logistic Regression	A simple and interpretable baseline model for binary classification that performs well on medical prediction problems and provides probabilistic outputs.
2	Random Forest	An ensemble learning algorithm that reduces overfitting through bagging and captures complex non-linear relationships between patient features.
3	Support Vector Machine	Effective for binary classification with structured data; capable of finding optimal decision boundaries using kernel methods.

Note: *If time allows, Decision Tree and XGBoost models may be evaluated for additional comparison.*

6. Evaluation Plan

Metrics

Accuracy — Proportion of correctly classified instances overall

Precision — Proportion of true positives among predicted positives — minimizes false alarms

Recall — Proportion of true positives among actual positives — minimizes missed diagnoses

F1-Score — Harmonic mean of Precision and Recall — primary selection criterion

ROC-AUC — Area under the receiver operating characteristic curve — tiebreaker metric

Confusion Matrix — Detailed breakdown of true/false positives and negatives for error analysis

Model Selection Criteria

The model with the highest F1-Score will be selected as the final model. **F1-Score balances Precision and Recall**, which is critically important for medical diagnosis where both false positives and false negatives carry significant consequences. If two models achieve similar F1-Scores, the model with the higher ROC-AUC Score will serve as the tiebreaker.

A comprehensive comparison table will be generated showing the performance of each algorithm across all metrics. Only the best-performing model will be serialized and deployed to production.

7. Deployment Sketch

FRAMEWORK FastAPI — lightweight, high-performance, auto-generates interactive API docs at /docs

ENDPOINT POST /predict

Request Payload

```
{
  "age": 54,
  "sex": 1,
  "cp": 0,
  "trestbps": 110,
  "chol": 206,
  "fbs": 0,
  "restecg": 0,
  "thalachh": 108,
  "exang": 1,
  "oldpeak": 0.0,
  "slope": 1,
  "ca": 2,
  "thal": 0
}
```

Response Payload

```
{
  "prediction": "Heart Disease",
  "probability": 0.91
}
```

The API will load the best saved model from [models/best_model.pkl](#) together with preprocessing artifacts (scaler, encoder) and return predictions for new patient data in real time.

8. Repository Plan

The project follows a clean architecture with clear separation of concerns:

```
heart-disease-prediction-api/
├── dataset/
│   └── heart.csv                # Raw dataset
├── src/
│   ├── preprocess.py           # Data cleaning & feature engineering
│   ├── train.py                # Model training & comparison
│   └── evaluate.py              # Metrics computation & reporting
├── api/
│   └── app.py                  # FastAPI application entry point
├── models/
│   ├── best_model.pkl          # Serialized best model
│   ├── scaler.pkl              # Fitted StandardScaler
│   └── encoder.pkl             # Fitted encoder (if applicable)
├── notebooks/
│   └── exploration.ipynb       # EDA & visualization
├── README.md                   # Project documentation
├── requirements.txt            # Python dependencies
└── project_paper.md            # Final project report
```